On the Adversarial Robustness of Locality-Sensitive Hashing in Hamming Space

MICHAEL KAPRALOV, EPFL, Switzerland
MIKHAIL MAKAROV, EPFL, Switzerland
CHRISTIAN SOHLER, University of Cologne, Germany

Locality-sensitive hashing (Indyk-Motwani'98) is a classical data structure for approximate nearest neighbor search. It allows, after a close to linear time preprocessing of the input dataset, to find an approximately nearest neighbor of any fixed query in sublinear time in the dataset size. The resulting data structure is randomized and succeeds with high probability for every fixed query independent of the randomness of the data structure. In many modern applications of nearest neighbor search the queries are, however, chosen *adaptively*. In this paper, we study the robustness of locality-sensitive hashing in Hamming space to adaptive queries. We present a simple adversary that can, under mild assumptions on the initial point set, provably find a query to the approximate near neighbor search data structure that the data structure fails on. Crucially, our adaptive algorithm finds the hard query exponentially faster than random sampling.

CCS Concepts: • **Theory of computation → Adversary models**; *Lower bounds and information complexity*; *Nearest neighbor algorithms*.

Additional Key Words and Phrases: sublinear algorithms, adversarial robustness, locality-sensitive hashing

ACM Reference Format:
Michael Kapralov, Mikhail Makarov, and Christian Sohler. 2025. On the Adversarial Robustness of Locality-Sensitive Hashing in Hamming Space. *Proc. ACM Manag. Data* 3, 2 (PODS), Article 102 (May 2025), 24 pages. https://doi.org/10.1145/3725239

1 Introduction

Nearest neighbor search is one of the most fundamental problems in data analysis. However, the development of efficient data structures for nearest neighbor search in high dimensions presents significant challenges, as finding the exact answer often necessitates a linear scan over the data [37]. Consequently, researchers have turned their attention to investigating approximate nearest neighbor search. One of the important techniques developed in this field is *locality sensitive hashing* (LSH), introduced in the seminal paper by Indyk and Motwani [24]. LSH is widely used in many applications including music retrieval [35], video anomaly detection [42], and clustering [28]. For a broader survey, see Jafari et al. [25].

The standard analysis of locality-sensitive hashing assumes that queries to the data structure are fixed in advance, i.e. future queries are independent of the answers to previous queries. This limitation restricts the usefulness of LSH, as real-world applications often involve adaptive querying. Thus, the understanding of the behavior of LSH in the adaptive setting is an important problem – and the focus of this paper. We set out to answer a very basic question:

Authors' Contact Information: Michael Kapralov, EPFL, Lausanne, Switzerland; Mikhail Makarov, EPFL, Lausanne, Switzerland; Christian Sohler, University of Cologne, Cologne, Germany.

ACM 2836-6573/2025/5-ART102
https://doi.org/10.1145/3725239

How quickly can an adversary interacting with the classical Indyk-Motwani LSH data structure find a bad query, i.e. a query that the data structure fails on?

Surprisingly, despite some recent work motivated by this question (LSH without false negatives [1, 33], discussed in detail in Section 2) and a flourishing line of work on the adversarial robustness of streaming and sublinear time algorithms (see, e.g, [7, 9–11, 14, 17, 18, 23, 40]), no such adversary was known prior to our work.

Our main result is an adversary, i.e. an algorithm for generating an adversarial query, with the following guarantees:

Theorem 1.1 (Informal version of Theorem 7.2). *Given an 'isolated point' z in a dataset P, one can find a point q at a distance at most r from z such that querying LSH with q returns no point. The number of queries to the LSH data structure needed to generate the point q is bounded by $O(\log(cr) \cdot \log(1/\delta))$, where δ is the failure probability of LSH.*

Notably, finding such a point through random sampling would require $\Omega(1/\delta)$ queries in expectation (see Theorem 3.3), which is exponentially slower in δ than the proposed adversary. We also evaluate our data structure experimentally to understand how the efficiency and effectiveness of our approach depends on the various parameters.

2 Related Work

Monte-Carlo Locality-Sensitive Hashing. Most of the work on Locality-Sensitive Hashing was done in the Monte-Carlo model, which we consider the standard model in this paper. That is, the goal is to construct a data structure with deterministically bounded query time that with high probability correctly answers the queries. Various locality-sensitive hash families are known for several different metrics, including Hamming distance [6, 24], ℓ_1 [20], the Euclidean metric [2], Jaccard similarity [13, 19], angular distance [3, 27, 31] and more. Asymmetric LSH solutions, which allow for trade-offs between query time and space, have received significant attention in the literature and are now well understood [5, 16, 26, 34]. Distance sensitive hash families, where collision probabilities are not necessarily monotone in the distance between points being hashed, have also been constructed [8]. Data-dependent results, which adapt the LSH family to the dataset to achieve better exponents, are also known [4, 5].

Las-Vegas Locality Sensitive Hashing without False Negatives. A different line of research aims to avoid false negatives by constructing LSH instances that answer queries deterministically correctly, but the running time is a random variable with guarantees given on its expectation. It was first shown by Pagh [33] that such a scheme exists, however this scheme for general parameter settings requires more space than the standard LSH. A later work by Ahle [1] achieves the same space-time trade-off as in [24]. Wei [38] further improves on it for ℓ_p, and offers a data-dependent construction following Andoni and Indyk [2]. Although Las Vegas constructions avoid the problem of having false negatives, their guarantees still hold only against an oblivious adversary. It is entirely possible that an adaptive adversary can find a sequence of queries with an average response time much larger than what is guaranteed against an oblivious adversary. In any case, investigation of such adversaries lies outside of the scope of this paper.

Robustness to adaptive adversaries. One of the main motivations behind this paper is to study the behavior of a classic data structure solving one of the closely linked problems of nearest neighbor, distance or norm estimation against an adaptive adversary. The robustness of existing constructions has been studied by [21, 22], which propose efficient attacks on the linear sketch for norm estimation and [17, 18], describing adaptive attacks on the CountSketch. At the same time, many new adversarially robust algorithms have been proposed, such as a deterministic

sketch constructions for a range of problems that work for all inputs [32], a robust version of the CountSketch algorithm [17], robust algorithm for ℓ_p-heavy hitters [39], an adversarially robust approach to solving approximate nearest neighbor in Hamming space without the use of LSH [29], a robust construction for vector ℓ_p-norm estimation [14], and adversarially robust streaming algorithms based on difference estimators for F_p-moment estimation and ℓ_2-heavy hitters [7].

Another fruitful line of work covers the use of differential privacy to make existing algorithms more robust against adaptive adversaries [9, 10, 15, 23, 40]. The main idea is to use differentially private mechanisms to hide the random bits of the algorithm from the adversary. This approach turned out to be applicable to a wide range of estimation problems.

3 Preliminaries

For $x > 0$, $\log x := \log_2 x$ will denote binary logarithm of x. For $n \in \mathbb{N}$, $[n] := \{1, \ldots, n\}$ will be the set of integers from 1 to n. For a point $p \in \{0,1\}^d$ and $i \in [d]$, p_i is the value of the point p along i-th dimension. For two points $p, q \in \{0,1\}^d$, $\mathrm{dist}(p,q) = \sum_{i \in [d]} |p_i - q_i|$ is their Hamming distance.

Basics of Locality Sensitive Hashing. Locality-sensitive hashing (LSH) is a method used to create randomized data structures for near-neighbor search, formalized as follows. A (randomized) data structure for the (r, cr)-approximate near neighbor problem in a metric space (X, dist) preprocesses a point set $P \subset X$ in such a way that for a query point $q \in X$ with probability at least $1 - \delta$ it (a) returns $p \in P$ with $\mathrm{dist}(p,q) \le cr$, if there is $p' \in P$ with $\mathrm{dist}(p',q) \le r$, (b) returns $\perp$, if there is no point $p \in P$ with $\mathrm{dist}(p,q) \le cr$, and (c) returns either $\perp$ or $p \in P$ with $\mathrm{dist}(p,q) \le cr$, otherwise. Here δ is the failure probability of the data structure. In this paper, we will consider the Hamming space, i.e. $X = \{0,1\}^d$ and dist is the Hamming distance and in the following discussion we focus on this setting.

The key component of LSH schemes is a locality-sensitive hash family. The idea behind them is that a function from such a family is more likely to map close points to the same bucket than far ones.

Definition 3.1 ([24]). *A hash family $\mathcal{H}$ is (r, cr, p_1, p_2)-sensitive if for every $p, q \in \{0,1\}^d$, the following holds. Let h be chosen uniformly at random from $\mathcal{H}$. If* $\mathrm{dist}(p,q) \le r$, *then* $\Pr[h(p) = h(q)] \ge p_1$. *Otherwise, if* $\mathrm{dist}(p,q) \ge cr$, $\Pr[h(p) = h(q)] \le p_2$.

For a d-dimensional Hamming space $\{0,1\}^d$, the locality-sensitive hash family has a very simple form [24]. Let $\mathcal{H} = \{h^{(i)}, i \in [d] : \forall p \in \{0,1\}^d, h^{(i)}(p) = p_i\}$ be the elementary hash function family. That is, it consists of hash functions $h^{(i)} : \{0,1\}^d \to \{0,1\}$, which project points from $\{0,1\}$ to their i-th bit. It is known that

Fact 3.2 ([24]). *For any c, r such that $r < cr \le d$, the elementary hash function family $\mathcal{H}$ is $(r, cr, 1 - \frac{r}{d}, 1 - \frac{cr}{d})$-sensitive.*

Indyk and Motwani give the following guarantee (assuming that the points come from a d-dimensional space):

Theorem 3.3 ([24]). *Suppose there exists a (r, cr, p_1, p_2)-sensitive family of hash functions. Then there is a data structure for the (r, cr)-approximate near neighbor problem (with $\delta = 1/2$) that uses $O(dn + n^{1+\rho})$ space and requires $O(n^\rho)$ evaluations of hash functions, where $\rho = \frac{\log(1-r/d)}{\log(1-cr/d)}$.*

In the following, we briefly describe the construction of an LSH data structure, which can be used to prove the above theorem. An LSH data structure consists of a multiset of G of L hash functions in the product $\mathcal{H}^k = (h_1, h_2, \ldots, h_k), h_j \in \mathcal{H}, j \in \{1, 2, \ldots, k\}$, i.e. $G \subseteq \mathcal{H}^k$ In other words, each element of G is constructed by sampling a sequence of k elementary hash functions $h_1, \ldots, h_k$

uniformly at random from $\mathcal{H}$ and then setting $g : \{0,1\}^d \to \{0,1\}^k$, $g(x) = (h_1(x), \ldots, h_k(x))$ to be their concatenation. That is, $\forall p \in \{0,1\}^d$, $g(p) = (h_1(p), \ldots, h_k(p))$. The intuition is that concatenating hash functions from $\mathcal{H}$ allows to exponentially increase the gap between probabilities p_1 and p_2 in such a way that if we sample L hash functions then two points p, q with distance at most r are likely to have at least one g_i with $g_i(p) = g_i(q)$ and we can recover the point. At the same time, the number of points with $\text{dist}(p,q) > cr$ and $g_i(p) = g_i(q)$ will be small, so that the recovery can be done efficiently.

The proof of Indyk and Motwani [24] essentially follows by choosing the parameters k and L appropriately, which is done as follows. We set $p_1 = 1 - r/d$ and $p_2 = 1 - cr/d$ such that $\mathcal{H}$ is (r, cr, p_1, p_2)-sensitive. We define $\rho = \frac{\log 1/p_1}{\log 1/p_2}$ and $\ell = n^\rho$. For an input set $P \subseteq \{0,1\}^d$ of n points we set $k = \lceil \log_{1/p_2} n \rceil$. For this setting of parameter, we get the guarantees provided in the above Theorem. In order to get a smaller failure probability we set $L = \lceil \lambda \ell \rceil$, which results in a failure probability of $1 - \exp(-\Omega(\lambda))$.

One particular property of the above LSH construction is the possibility of incorrectly returning that there is no near point (i.e. returning $\perp$). Such an answer is called a *false negative*.

Definition 3.4 (False negative)**.** *Consider an LSH data structure initialized for the point set $P \subseteq \{0,1\}^d$. We say that the answer to a query $q \in \{0,1\}^d$ is a **false negative** query for this LSH instance if $\textsc{QueryLSH}(q, G) = \perp$, but there exists a point $p \in P$ such that* $\text{dist}(p,q) \le r$.

Defining the adversary. The aim of this paper is to study how one can force LSH to return a false negative through adaptive queries. We take the role as an *adversary* and our goal is to design an *adversarial algorithm* that has access to an LSH instance and forces it to return a false negative. In particular, we provide guarantees on the number of queries required until LSH makes a false negative.

We now describe our formal setting. Our LSH instance is constructed according to the scheme in Section 3. Recall that this LSH scheme samples a multiset G of L hash functions $g_1, \ldots, g_L$ uniformly at random from the set of allowed hash functions of the form $g : \{0,1\}^d \to \{0,1\}^k$ with $g(p) = (h_1(p), \ldots, h_k(p))$ for elementary hash function h_j. The LSH scheme receives as input a point set $P \subseteq \{0,1\}^d$ of size n and parameters n, d, c, r and λ and calculates ℓ, ρ, L and k as described earlier. Our adversarial algorithm may use the input point set P, its size n and dimension d, and the parameters r, c and λ of the LSH scheme and can interact with the LSH data structure using Algorithm 1 described below. That is, the LSH data structure is in a black box and the only interaction uses Algorithm 1. In particular, the randomness used to construct the LSH instance and so the hash functions is unknown to the adversarial algorithm.

Algorithm 1 QueryLSH(q, r, c)

$S \leftarrow \emptyset$
for all $g \in G$ **do**
  $S = S \cup \{p \in P : g(p) = g(q) \text{ and } \text{dist}(p,q) \le cr\}$
return any point in S, or $\perp$ if S is empty

Note that in an LSH implementation one would store all buckets of g in a hash table. This way, line 3 can be implemented without going through all points. We also remark that since we do not have control of the parameters and point set our adversarial algorithm cannot be expected to work in all settings (for example, if the number of hash functions is very large, then no false negatives exist). We summarise the aim of the adversary as follows:

Definition 3.5 (Adversarial attack on LSH)**.** *Given access to a* QueryLSH *function for a fixed instance of Locality-Sensitive Hashing algorithm, make as few as possible adaptive queries to it such that the last query is a false-negative one.*

For an LSH instance with failure probability δ, a random sampling approach would take around $1/\delta$ queries to find a false negative. Our adversarial algorithm presented here can do so exponentially faster, reducing the number of queries to be proportional to $\log(1/\delta)$.

4 Adversarial Algorithm

Define the set $\textsc{Coll}(p, q) = \{g \in G : g(p) = g(q)\}$ to be the set of hash functions in G that collide on p and q. Our algorithm starts with finding a point z in the dataset such that any other point is at least at a distance of $2cr$ from z. We assume that such a point exists, and we experimentally explore the case when it doesn't in Section 6. We also show that this assumption is reasonable for a randomly chosen point set in Lemma 7.3. We refer to z as **origin point**.

Algorithm outline. The idea of the algorithm is to query a random point q at distance $r - t$ from z such that the expected number of hash functions hashing q to the same bucket as z is at most $t/2$. We show that this is true when $t = 2e^2(\lambda + 1)$, which is the value we use throughout this section. This implies that with a constant probability $|\textsc{Coll}(q, z)| \leq t$. The algorithm now does t iterations and in each of them it moves point q away from z by flipping one bit in it in such a way that the size of the set $\textsc{Coll}(q, z)$ also decreases by at least 1.

To find this bit, the algorithm selects $cr - \text{dist}(q, z)$ random positions where q and z are equal and flips those bits in q, resulting in a new point $\tilde{q}$. $\tilde{q}$ is located at distance cr from z, so with a high probability $\textsc{QueryLSH}(\tilde{q}) = \perp$. Now consider a **path** between q and $\tilde{q}$: a sequence of points starting with q and ending with $\tilde{q}$, where each two consecutive points differ in one bit and where each point is further away from z than its predecessor. Because $\tilde{q}$ doesn't hash with z but q does, there are two consecutive points q', q'' on this path with the same property. Consider the bit where they differ. Because $\textsc{Coll}(q', z) \neq \emptyset$ and $\textsc{Coll}(q'', z) = \emptyset$, this bit must lie in the support of all functions in $\textsc{Coll}(q', z)$.[1] Therefore, by flipping it in q the algorithm removes these functions from $\textsc{Coll}(q, z)$. It is easy to see that at the end we get a point q at a distance at most r from z such that $\textsc{Coll}(q, z) = \emptyset$, so $\textsc{QueryLSH}(q) = \perp$ and q is a false negative.

In the following we prove Lemmas 4.5 and 4.7, which are the key results underpinning the correctness of our algorithm. Lemma 4.5 essentially shows that a point q sampled at distance $r - t$ collides with z in at most $O(t)$ hashings with constant probability. Lemma 4.7 that says that if we move this point randomly and monotonically away from z to distance cr, it will not hash at all with z. A key property of these lemmas is that, unlike in the standard analysis of LSH, we fix the hash functions and sample points at random. This gives us a different perspective on how LSH behaves when queried with random points, which reflects the fact that our goal is to construct an adversary for LSH.

First, we state intermediate results bounding the size of k and the support of hash functions.

Lemma 4.1 (Bounds on k)**.** *If $cr/d \leq 1/5$ and $n > e$, one has $2\frac{d}{cr}\ln n \geq k \geq 0.5\frac{d}{cr}\ln n$.*

The proof is deferred to Section 7.

[1] For an elementary hash function $h^{(i)} \in \mathcal{H}$ its **support** is defined as the set $\{i\}$. For a hash function $g \in G$, $g = (h_1, \ldots, h_k)$ its **support** is the union of supports of $h_1, \ldots, h_k$.

Algorithm 2 Adaptive adversarial walk from z.

Input: Origin point z, parameters r, c, and λ

$t \leftarrow 2e^2(\lambda+1)$
Sample q uniformly at random from all points at distance $r-t$ from z
while QueryLSH$(q) = z$ **do**
  if $\mathrm{dist}(q,z) \geq r$ **then**
    return $\perp$
  $q' \leftarrow q$
  while QueryLSH$(q') = z$ **do**
    Let $I = \{i \in [d] : q'_i = z_i\}$
    Select $j \in I$ uniformly at random
    Set $q'_j = 1 - q'_j$
    if $\mathrm{dist}(q',z) > cr$ **then**
      return $\perp$
  Set $q_j = 1 - q_j$
return q

Fact 4.2 (Chernoff bound). *Let $X_1, \ldots, X_n$ be independent Bernoulli random variables. Let $S = \sum_{i=1}^n X_i$. Let $\delta > 0$. Then*

$$\Pr[S \geq (1+\delta)\,\mathbb{E}(S)] \leq \exp\left(-\frac{\delta^2\,\mathbb{E}(S)}{2+\delta}\right).$$

Lemma 4.3. *Suppose that a hash function g was constructed according to the procedure outlined in Section 3 and $n > e$. Then with probability at least $1 - 1/n^3$ the size of the support of g is at least $k - 7\log n \cdot \max\{1, \frac{k(k-1)}{2d}\}$.*

Proof. Recall that a hash function g is composed of hash functions $h_1, \ldots, h_k$ each of which are supported on one dimension from $\{1, \ldots, d\}$. The dimension on which each of the function h_i is supported is picked uniformly and independently from $\{1, \ldots, d\}$. We will call hash function h_i duplicated if for some $j < i$, $h_i = h_j$. Note that the first instance of each hash function in the range according to this definition is not duplicated. Then, because $k = |\text{support of } g| + \#\text{duplicated hash functions } h$, it is enough to bound the number of duplicated hash functions to conclude the statement of the lemma, which we will denote by S.

Consider the following equivalent process of constructing g. We will pick each of the h_i in a sequence of increasing i. Denote by t_i the size of the union of supports of $h_1, \ldots, h_{i-1}$. For each i we will first reorder the range $\{1, \ldots, d\}$ so that the first t_i entries would be the dimension indices already in the support of $h_1, \ldots, h_{i-1}$, and the rest would be everything else. We will then pick a number $a_i \in [d]$ uniformly at random, and this number would indicate the position of the dimension index in the new range on which h_i will be supported on.

It is now easy to see that the hash function h_i is duplicated iff $a_i \leq t_i$.

Since the support of first $i-1$ hash functions is at most of size $i-1$, it always holds that $t_i \leq i-1$. Let σ_i be an indicator of the event $a_i \leq t_i$, and ξ_i be an indicator of the event $a_i \leq i-1$. Then $\sum_{i=1}^k \sigma_i = S$ and for each i, $\sigma_i \leq \xi_i$.

Hence we will upper bound S by $S' := \sum_{i=1}^k \xi_i$, and upper bound S' using the Chernoff bound. Indeed $\mathbb{E}(S') = \sum_{i=1}^k \mathbb{E}(\xi_i) = \sum_{i=1}^k \frac{i-1}{d} = \frac{k(k-1)}{2d}$. Note that $\delta^2/(2+\delta) \geq \delta/2$ when $\delta > 2$, which is the case for the values we use below.

Now, we first consider the case when $\frac{k(k-1)}{2d} \geq 1$.

By applying Fact 4.2 to ξ_i and setting $\delta = 6 \ln n$, we get

$$\begin{aligned}\Pr\left[S' \geq 7 \ln n \frac{k(k-1)}{2d}\right] &\leq \Pr\left[S' \geq (1+6\ln n)\frac{k(k-1)}{2d}\right] \\ &\leq \exp\left(-\frac{36\ln^2 n}{2+6\ln n} \cdot \frac{k(k-1)}{2d}\right) \\ &\leq \exp\left(-3\ln n \frac{k(k-1)}{2d}\right) \\ &\leq 1/n^3.\end{aligned}$$

When $\frac{k(k-1)}{2d} < 1$, set $\delta = 6 \ln n \frac{2d}{k(k-1)}$.

$$\begin{aligned}\Pr\left[S' \geq 7\ln n\right] &\leq \Pr\left[S' \geq (1+6\ln n \frac{2d}{k(k-1)})\frac{k(k-1)}{2d}\right] \\ &\leq \exp\left(-3\ln n \frac{2d}{k(k-1)} \cdot \frac{k(k-1)}{2d}\right) \\ &\leq \exp\left(-3\ln n\right) \\ &\leq 1/n^3.\end{aligned}$$

Therefore,

$$\begin{aligned}&\Pr\left[|\text{support of } g| < k - 7\log n \cdot \max\left\{1, \frac{k(k-1)}{2d}\right\}\right] \\ &\qquad = \Pr\left[S \geq k - 7\log n \cdot \max\left\{1, \frac{k(k-1)}{2d}\right\}\right] \\ &\qquad \leq \Pr\left[S' \geq k - 7\log n \cdot \max\left\{1, \frac{k(k-1)}{2d}\right\}\right] \\ &\qquad \leq 1/n^3,\end{aligned}$$

which concludes the proof. □

Lemma 4.4 (Support lower bound)**.** *Let $\lambda \leq n$ and $n > e$. Then with probability at least $1 - \frac{1}{n}$ the size of the support of each of the functions $g \in G$ is at least $k - 7\ln n \cdot \max\{1, \frac{k^2}{2d}\}$.*

Proof. By Lemma 4.3, the probability that the support of any fixed hash function in G is at least $k - 7\log n \cdot \max\{1, \frac{k(k-1)}{2d}\}$ is at least $1 - 1/n^3$. The number of hash functions in G is $\lceil \lambda n^\rho \rceil \leq n^2$ and so by the union bound, the probability that there exists a hash function $g \in G$ with support less than $k - 7\log n \cdot \max\{1, \frac{k^2}{2d}\}$ is at most $\lceil \lambda n^\rho \rceil \cdot \frac{1}{n^3} \leq 1/n$. □

The next two lemmas lower bound the size of the support of hash functions in LSH when they are chosen as described in Section 3. We now show the first lemma, which bounds the number of hash functions hashing a point at distance $r - t$ from z in the same bucket with z. We outline the proof below, and present the full proof in Section 7.1.

Lemma 4.5. *Let $n > e$, $8e^2(\lambda+1)\ln n \leq cr$, $28\ln^3 n/c^2 \leq r \leq d/(14\ln n)$, $r \leq d/(5c)$ and $c < \ln n$. Let the hash functions G be such that the size of the support of each is at least $k - 7\ln n \cdot \max\{1, \frac{k^2}{2d}\}$. Let $t = 2e^2(\lambda+1)$. Let q be a point at distance $r - t$ from z chosen uniformly at random. The expected size of $\textsc{Coll}(q, z)$ is at most $e^2(\lambda+1)$, where the expectation is taken with respect to the randomness of q.*

Proof. We fix an arbitrary set G of L hash functions that satisfy the condition of the lemma. By linearity of expectation,

$$\mathbb{E}[|\textsc{Coll}(q,z)|] = \sum_{g\in G} \Pr[g(q) = q(z)],$$

where the randomness is over the choice of q as in the lemma.

Let $x = 7\ln n \cdot \max\{1, \frac{k^2}{2d}\}$

In the following we consider an arbitrary $g \in G$ and derive an upper bound for the collision probability of q and z.

$$\Pr_{q\sim UNIF(\mathbb{S}(z,r-t))}[g(q) = g(z)] \le \left(1 - \frac{k-x}{d}\right)^{r-t}, \tag{1}$$

where $\mathbb{S}(z, r-t) = \{p \in \{0,1\}^d : \mathrm{dist}(z,p) = r-t\}$.

To see it, consider the following way to sample q: first sample $r-t$ bits uniformly at random with replacement from $[d]$ to form a set B, $|B| \le r-t$, and then flip the bits from B in z to obtain a point q'. After that choose the rest of $r - t - \mathrm{dist}(z,q')$ bits without replacement uniformly at random among the bits $[d] \setminus B$ and flip them. Because the bits flipped in the second step are different from the bits flipped in the first step, if $g(q') \neq g(z)$ then $g(q) \neq g(z)$. Therefore, $\Pr[g(q') = g(z)] \ge \Pr[g(q) = g(z)]$.

On the other hand, $g(q') = g(z)$ if and only if all of the bits sampled into B do not lie in the support of g. Because the support of g is at least $k - x$, each sampled bit doesn't lie in it with probability at most $1 - \frac{k-x}{d}$. Since the bits are chosen independently, we have $\Pr[g(q') = g(z)] \le (1 - \frac{k-x}{d})^{r-t}$. This establishes (1).

We are now in a position to upper bound the expected size of $\textsc{Coll}(q,z)$, i.e. the number of hashings that q and z collide under, obtaining the result of the lemma. Using (1) we get

$$\mathop{\mathbb{E}}_{q}[|\textsc{Coll}(q,z)|] = \mathop{\mathbb{E}}_{q}\left[\sum_{g\in G} I(g(q) = g(z))\right] = \sum_{g\in G}\Pr[g(q) = g(z)] \le L \cdot \left(1 - \frac{k-x}{d}\right)^{r-t}. \tag{2}$$

Recall that by our parameter setting the number of hashings L satisfies $L = \lceil \lambda \ell \rceil \le (\lambda+1)\ell$, where

$$\ell = n^{\rho} = \left(1 - \frac{r}{d}\right)^{-\log_{1/p_2} n} \le \left(1 - \frac{r}{d}\right)^{-k}. \tag{3}$$

Hence, by (3), we have the following upper bound on the expectation.

$$\mathop{\mathbb{E}}_{q}[|\textsc{Coll}(q,z)|] \le (\lambda+1)\left(1 - \frac{r}{d}\right)^{-k}\left(1 - \frac{k-x}{d}\right)^{r-t} \tag{4}$$

The only step left is to upper bound the right hand side of (4). Therefore, we need to upper bound $\left(1 - \frac{r}{d}\right)^{-k}\left(1 - \frac{k-x}{d}\right)^{r-t}$. We have by Fact 4.6

$$\begin{aligned}
(1 - \frac{k-x}{d})^{r-t} &\le \exp(-\frac{k-x}{d})^{r-t} \\
&= \exp(-\frac{kr}{d} + \frac{x(r-t)}{d} + \frac{tk}{d}) \\
(1 - \frac{r}{d})^{k} &\ge \exp(-\frac{r}{d} + \frac{r^2}{d^2})^{k} \\
&= \exp(-\frac{kr}{d} - \frac{kr^2}{d^2})
\end{aligned}$$

The largest term $\frac{kr}{d}$ cancels out in the fraction, so we are only left with a sum of smaller terms.

$$\frac{(1-\frac{k-x}{d})^{r-t}}{(1-\frac{r}{d})^k} \leq \exp\left(\frac{tk}{d} + \frac{kr^2}{d^2} + \frac{x(r-t)}{d}\right). \tag{5}$$

Now we will establish upper bounds on each of the summands in the right-hand side of (5). First, by Lemma 4.1,

$$k \leq 2\frac{d}{cr}\ln n.$$

This implies:

- $k \leq d/(2t)$, since $8e^2(\lambda+1)\ln = 4t\ln < cr$, so $2\frac{d}{cr}\ln n < d/(2t)$.
- $k \leq d^2/r^2$, since $r \leq d/(14\ln n) \leq \frac{dc}{2\ln n}$, so $2\frac{1}{c}\ln n \leq d/r$ and $2\frac{d}{cr}\ln n \leq d^2/r^2$.
- $k^2 \leq \frac{d^2}{7r\ln n}$, since $r \geq 28\ln^3 n/c^2$, so $\frac{1}{7\ln n} \geq 4\frac{1}{c^2 r}\ln^2 n$ and $4\frac{d^2}{c^2r^2}\ln^2 n \leq \frac{d^2}{7r\ln n}$

The last inequality implies that $7\ln nr \cdot \frac{k^2}{2d^2} \leq 1/2$, and $r \leq \frac{d}{14\ln n}$ implies $7\ln nr/d \leq 1/2$, so

$$\frac{x(r-t)}{d} \leq xr/d \leq 1/2.$$

On the other hand, the first two bounds on k imply $tk/d < 1/2$ and $kr^2/d^2 < 1$, which finally allows us to bound rhs of (5):

$$\exp\left(\frac{tk}{d} + \frac{kr^2}{d^2} + \frac{x(r-t)}{d}\right) \leq \exp(0.5+1+0.5) = e^2.$$

Putting the above together with (4) we therefore conclude that

$$\mathop{\mathbb{E}}_{q}[\textsc{Coll}(q,z)] \leq e^2(\lambda+1)$$

as required. □

We now prove the second lemma, which bounds the probability that a point at distance cr will be hashed with the origin. We will need the following fact, with it's proof deferred to Section 7.1.

Fact 4.6. *For $x \in [0,+\infty)$, $1-x \leq e^{-x}$. For $x \in [0,1/5]$, $e^{-x-x^2} \leq 1-x$.*

Lemma 4.7. *Let $n > e$, $cr/d \leq 1/5$, $c \geq 1 + \frac{80+16\ln(\lambda+1)}{\ln n}$, $8e^2(\lambda+1) \leq n^{1/8}$. Let G be such that no hash functions $g \in G$ has support smaller than $k/2$. Let q be a point at distance at most r from z such that $|\textsc{Coll}(q,z)|$ is at most $2e^2(\lambda+1)$. Let q' be a point chosen uniformly at random among all of the points at distance cr from z such that for all $i \in [d]$, if $q_i \neq z_i$, then $q'_i \neq z_i$. Then with probability at least $1 - 1/(32e^2(\lambda+1))$, $\textsc{Coll}(q',z) = \emptyset$, i.e. no function g hashes q' and z to the same bucket.*

Proof. Fix $g \in \textsc{Coll}(q,z)$. First notice that q' defined in lemma statement can be generated as follows. One first selects a uniformly random subset S of $cr - \mathrm{dist}(q,z)$ coordinates in $I = \{i \in [d] : q_i = z_i\}$. Then one lets $q'_i = q_i \oplus 1$ (negation of q_i) for $i \in S$ and $q'_i = q_i$ for $i \in [d] \setminus S$. We denote this distribution by $\mathcal{D}(q)$. Note that because $g(q) = g(z)$ (since $g \in \textsc{Coll}(q,z)$ by assumption), the support of g is a subset of I. We will show that $\Pr_{q'\sim\mathcal{D}(q)}[g(q') = g(z)] \leq \left(1 - \frac{k}{2d}\right)^{(c-1)r}$. In order to establish the above, we view the generation of q' as a two step process. First we uniformly sample $cr - \mathrm{dist}(q,z) \geq cr - r$ bits from I *with replacement* into a set B (removing duplicates) and flip them in q – denote the result by $\tilde{q}$. Next, we choose $cr - \mathrm{dist}(\tilde{q},z)$ more bits from $I \setminus B$ and flip them in $\tilde{q}$ to get q'. It again holds that $g(\tilde{q}) \neq g(z)$ implies $g(q') \neq g(z)$, so $\Pr_{q'}(g(q') \neq g(z)) \geq \Pr_{\tilde{q}}(g(\tilde{q}) \neq g(z))$.

The probability that each bit chosen in the first step lies in support of g is at least $\frac{k/2}{d-\mathrm{dist}(q,r)} \geq \frac{k}{2d}$. Hence, the probability that none of them lie in the support, i.e. $\Pr_{\tilde{q}}(g(\tilde{q}) = g(z))$, is at most $(1-\frac{k}{2d})^{cr-r}$. This establishes the inequality.

By Lemma 4.1, $k \geq 0.5\frac{d}{cr}\ln$. By applying Fact 4.6 to $(1-\frac{k}{2d})^{cr-r}$, we obtain: $\Pr[g(q') = g(z)] \leq \left(1-\frac{k}{2d}\right)^{(c-1)r} \leq n^{-\frac{c-1}{4c}}$. Now consider two cases. First, if $c \geq 2$, then the rhs of the above inequality satisfies $n^{-\frac{c-1}{4c}} \leq n^{-1/4} \leq \frac{1}{(8e^2(\lambda+1))^2}$ since $8e^2(\lambda+1) \leq n^{1/8}$ by assumption of the lemma. Otherwise, using that $c \geq 1 + \frac{80+16\ln(\lambda+1)}{\ln n}$ by the assumption of the lemma, we get $\frac{c-1}{4c} \geq \frac{c-1}{8} \geq \frac{10+2\ln(\lambda+1)}{\ln n} \geq \ln(8e^2(\lambda+1))/\ln n$. Thus, in both cases one has $n^{-\frac{c-1}{4c}} \leq \frac{1}{(8e^2(\lambda+1))^2}$. Finally, since $|\textsc{Coll}(q,z)| \leq 2e^2(\lambda+1)$ by the assumption of the lemma, by union bound across all $g \in \textsc{Coll}(q,z)$, $\Pr[\textsc{Coll}(q',z) \neq \emptyset] \leq 1/(32e^2(\lambda+1))$, as required. □

4.1 Analysis of the algorithm

Equipped with Lemma 4.5 and Lemma 4.7, we are now ready to analyse Algorithm 2. Throughout this section we make a strong assumption that the input point z is **isolated**[2]. That is, the distance from z to any other point in P that is not equal to z is at least $2cr$. This means that as long as we only make queries q at distance at most cr to z, the only two possible results of QueryLSH are z and $\perp$. This in turn means that $\textsc{QueryLSH}(q) = z$ iff $\textsc{Coll}(q,z) \neq \emptyset$.

First, we analyze the inner loop of the algorithm — lines 6 to 12. Its goal is to find a bit to flip in q such that the size of $\textsc{Coll}(q,z)$ would decrease afterward. Of course, our algorithm only gets black-box access to QueryLSH, and cannot test the size of $|\textsc{Coll}(q,z)|$ in general. However, for a point q' one can, using black-box access to QueryLSH, distinguish between $|\textsc{Coll}(q',z)| = 0$ and $|\textsc{Coll}(q',z)| > 0$: due to the assumption that z is an isolated point we have $|\textsc{Coll}(q',z)| > 0$ iff $\textsc{QueryLSH}(q',z) \neq \perp$. As we show, this ability suffices. Algorithm 2 simply keeps flipping bits in q at random (but restricted to coordinates where q and z agree) until z is no longer returned by QueryLSH on the modified query: the last flipped bit must lead to a decrease in $|\textsc{Coll}(q,z)|$ even in the original query q! More formally, our algorithm finds two points q' and q'' such that q' lies on a path[3] between q'' and q, and $\textsc{Coll}(q'',z) = \emptyset$ and $\textsc{Coll}(q',z) \neq \emptyset$, and that differ only in one bit. This is enough to conclude that this bit lies in support of all hash functions of $\textsc{Coll}(q',z)$, so by flipping it in q we would remove all of those hash functions from $\textsc{Coll}(q,z)$.

Lemma 4.8 (Inner loop of the adversarial walk). *Let $n > e$, $cr/d \leq 1/5$, $c \geq 1 + \frac{80+16\ln(\lambda+1)}{\ln n}$, $8e^2(\lambda+1) \leq n^{1/8}$. Let G be such that no hash functions $g \in G$ has support smaller than $k/2$. Suppose that the point z passed to Algorithm 2 is located at distance at least $2cr$ from any other point in P. Let q be sampled in line 2 be such that $|\textsc{Coll}(q,z)| \leq 2e^2(\lambda+1)$. For a fixed iteration of the outer loop in line 3 of Algorithm 2, with probability at least $1 - 1/(32e^2(\lambda+1))$ the inner loop between lines 6 and 12 finds two points q' and q'' such that $\textsc{QueryLSH}(q') \neq z$, $\textsc{QueryLSH}(q'') = z$, and q' and q'' differ in one bit.*

Proof. Fix an iteration of the outer loop. Then the inner loop at line 7 essentially does a random walk away from z, always increasing the distance from z after every flip. There are two possibilities: either it finds a point at distance at most cr from z such that $\textsc{QueryLSH}(q) \neq z$, i.e. $\textsc{Coll}(q',z) = \emptyset$, or it reaches a point q'' at distance cr such that $\textsc{QueryLSH}(q'') = z$, where the algorithm would return $\perp$.

[2]We provide some evidence that this assumption might be necessary in Appendix A.

[3]Recall that a **path** between a and b is a sequence of points starting with a and ending with b, where each two consecutive points differ in one bit and where each point is further away from a than its predecessor.

Assume that the latter happens. Since the bits flipped are chosen at random, this process is equivalent to flipping $cr - \mathrm{dist}(q, z)$ different bits of q where q is not equal to z. This is the same process as in statement of Lemma 4.7, so by this lemma the probability that this case takes place is at most $1/(32e^2(\lambda + 1))$. □

We are now ready to analyze the full algorithm. We propose two versions of it, Algorithms 2 and 3. The only difference between them is in the inner loop, which allows Algorithm 3 to use less queries. Although we state the theorem for both Algorithms 2 and 3, we only present the analysis for Algorithm 2 here. The proof for Algorithm 3 can be found in Section 7.2.

Theorem 4.9. *Let $n > e$, $28\ln^3 n \le r \le d/(28\ln n)$, $\lambda \le \frac{1}{8e^2}\min\{\frac{r}{\ln n}, n^{1/8}\} - 1$, and $1 + \frac{80+16\ln(\lambda+1)}{\ln n} \le c \le \ln n$. Let $t = 2e^2(\lambda + 1)$ be the parameter from Algorithm 2 (Algorithm 3). Suppose that the point z passed to Algorithm 2 (Algorithm 3) is located at a distance of at least $2cr$ from any other point in P. With probability at least $1/4 - 1/n$ the algorithm finds a point q at a distance of at most r from z such that querying LSH with q returns no point. It uses at most $O(cr \cdot \lambda)$ ($O(\log(cr) \cdot \lambda)$ for Algorithm 3) queries to the LSH data structure.*

Proof for Algorithm 2. First we show that under this parameter setting the requirements of Lemmas 4.5 and 4.7 are satisfied. For this we need to show that $r \le d/(5c)$ and that the support of each hash function $g \in G$ is at least $k/2$.

The inequality $r \le d/(5c)$ immediately follows from $c \le \ln n$ and $r \le d/(28\ln)$.

By Lemma 4.4, each hash function $g \in G$ has support at least $k - 7\ln n \cdot \max\{1, \frac{k^2}{2d}\}$ with probability at least $1/n$. By Lemma 4.1 and the parameter setting, $r \le d/(28\ln n) \le d/(28c)$, hence $14\ln n \le \frac{d}{2cr}\ln n \le k$. On the other hand, $r \ge 28\ln^3 n \ge 14\ln n/c$, so $k \le \frac{2d}{cr}\ln n \le \frac{d}{7\ln}$ and $\frac{k^2}{2d} \le \frac{k}{14\ln n}$. This means that $k/2 \ge 7\ln n \cdot \max\{1, \frac{k^2}{2d}\}$, so each $g \in G$ has support at least $k/2$ with probality at least $1 - 1/n$.

To conclude the theorem statement, we show in this proof that Algorithm 2 returns a point q within distance r of z such that $\textsc{Coll}(q, z) = \emptyset$.

Let q be the point sampled in line 2. By Lemma 4.5 and Markov's inequality, $|\textsc{Coll}(q, z)| \le 2e^2(\lambda + 1)$ with probability $1/2$. Conditioned on those inequalities holding, we will show that with a probability at least $1 - \frac{1}{8e^2(\lambda+1)}$ the algorithm finds a bit i during each execution of the outer loop at line 3 such that, after flipping this bit in q, $|\textsc{Coll}(q, z)|$ decreases by at least 1. This would mean that after $2e^2(\lambda + 1)$ iterations $\textsc{Coll}(q, z) = \emptyset$. This happens with probability at least $1 - (1/2 + 2e^2(\lambda + 1)\frac{1}{8e^2(\lambda+1)}) = 1/4$ by a union bound.

By taking the union bound with the probability that each $g \in G$ has support at least $k - 4\log n$, we obtain the final success probability of at least $1/4 - 1/n$.

The inner loop at line 6 by Lemma 4.8, with probability at least $1 - 1/(32e^2(\lambda + 1))$, finds two points q' and q'' such that $\textsc{Coll}(q'', z) \ne \emptyset$, $\textsc{Coll}(q', z) = \emptyset$ and q', q'' differ in exactly one bit.

Assuming such a pair of points is found, let i be the position of the bit that is different between them. Since $\textsc{Coll}(q'', z) \ne \emptyset$, there is a function $g \in \textsc{Coll}(q, z)$ such that $g(q'') = g(z)$. On the other hand, because $\textsc{Coll}(q', z) = \emptyset$, $g(q') \ne g(z)$, therefore bit i must lie in the support of g. Hence after we flip this bit in q, $g(q) \ne g(z)$ and $|\textsc{Coll}(q, z)|$ decreases.

Note that because we are doing the inner loop at most t times, $\mathrm{dist}(q, z) \le r$, so the prerequisites for Lemma 4.8 always hold.

The outer loop is executed at most $2e^2(\lambda+1)$ times, and the inner loop makes at most $cr - (r - t)$ iterations, as each iteration moves q' further away from z. Therefore, the total number of queries is bounded by $O((cr - r + t)\lambda)$. □

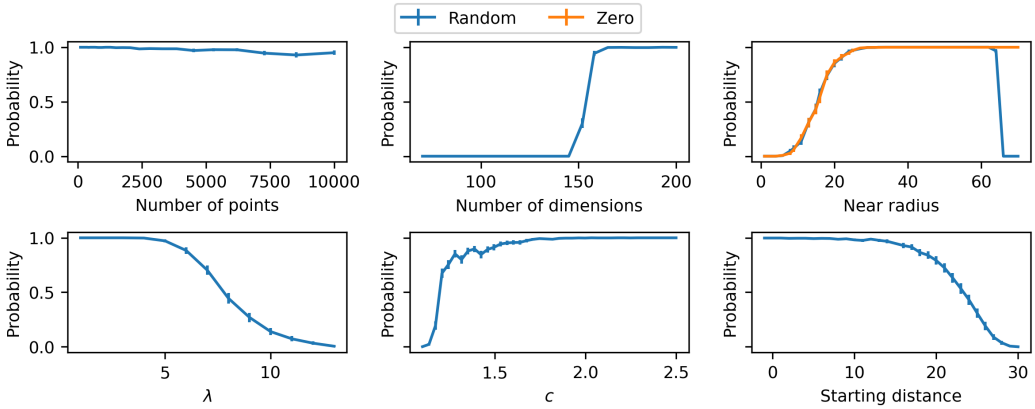

Fig. 1. Dependence of success probability on various parameters. All experiments are done on the Random dataset, with the third one also featuring the Zero dataset.

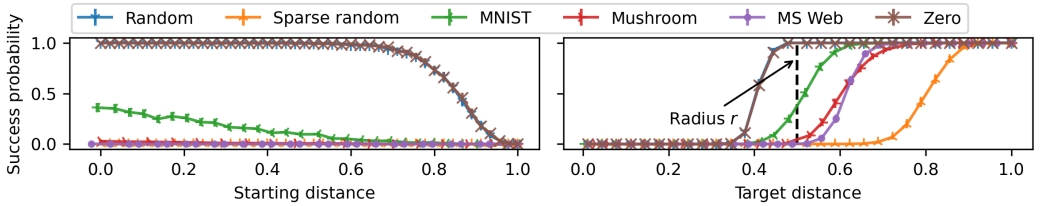

Fig. 2. Dependence of the success probability on the value of t and on the value of desired distance of false negative query from the origin.

5 Discussion of potential defenses against our attacking scheme

As was mentioned in Section 2, there already exists large literature on the robustification of LSH, many of which work against our adversary. In particular, we highlight Ahle [1], which provides a false-negative-free Las-Vegas type data structure with query time $dn^{1/c+o(1)}$ and used space $dn^{1+1/c+o(1)}$, achieving the same asymptotic performance as Indyk and Motwani [24], as well as Cherapanamjeri and Nelson [14], which can be applied to LSH to guarantee perfect robustness by paying $O(d)$ times more space.

However, both approaches are not perfect within the same parameter bounds, as approach of Ahle [1] doesn't give any guarantees on query time in this setting, and the space usage of Cherapanamjeri and Nelson [14] is too prohibitive in settings with large d. We experimentally investigate how the latter, and a construction based on differential privacy, fares against our adaptive adversary when the available space is limited in Experiment 6 in Appendix B. Interestingly, these defences perform well in this setting, which suggests that they might be reasonably effective even against a general adversary.

6 Experiments

The goal of this section is to verify the efficacy of our adversary on real datasets, where an isolated point does not necessarily exist, and for parameter configurations outside of the guarantees of Theorem 4.9.

Implementation details. Our implementation of LSH follows Algorithm 1 in Section 3 [24]. To summarise, given a query point q, we iterate through all points in the dataset that hash together with q under some hash function, then return the first one which lies at distance at most cr from q, and return nothing if there is no such point. The adversary is implemented according to Algorithm 3.

Experiment 1: Influence of parameters on success probability. The goal of this experiment is to explore within which parameter setting the algorithm works. For this experiment we use synthetic datasets, since for them we can easily vary the number of points and dimensions. We use two datasets:

Zero. This dataset entirely consists of n copies of the all-zero point. This dataset is the easiest for our algorithm to find a false negative in, as it is equivalent to a dataset containing only a single point, which would automatically be isolated.

Random. Each point is sampled i.i.d. uniformly, with each bit having a $1/2$ probability to be set to 1.

For each point on the plots, the experiments were run 1000 times, and the result is the mean of the associated values. Plots contain error bars computed as standard error of the mean, but sometimes they are too small to be visible.

The parameters that we vary are n, d, r, λ, c and t. Each parameter is changed individually. The default setting of parameters is $n = 1000$, $d = 300$, $\lambda = 4$, $r = 30$, $c = 2$. The origin point z is picked at random. The offset t is set to r, or, in other words, the initial point q is initialized to be equal to z. This is different than what was used in Algorithms 2 and 3. However, the intuition is that if instead of setting q to be a random point at distance $2e^2(\lambda + 1)$ from z we set $q = z$ and then move it away from z in a way that ensures that the size of $\textsc{Coll}(q, z)$ decreases for every changed bit, the probability that the algorithm will find a false negative query would only increase. This intuition ends up being supported by the experiment results.

The results are presented in Figure 1. The sixth subplot, counting left-to-right and top-to-bottom, supports the claim that we just made: the lower the starting distance of the point q, which is equal to $r - t$, the higher the success probability of the algorithm. The first and fifth subplots are straightforward. The number of points n has a very weak effect on the success probability. Parameter c also has a weak influence if it is large enough, and when it is small it is unlikely that a point at distance cr does not hash with z, see Lemma 4.7. The drop in probability on the left-hand side of the third subplot where the near radius r is varied and the right-hand side of the fourth where λ is varied can be explained by the fact that a random point at distance r has in expectation $\Omega(\lambda)$ hash functions hashing it with z. Hence, intuitively, for it to be possible to eliminate all collisions with z, it must be that $t \geq \Omega(\lambda)$, which also implies that $r \geq \Omega(\lambda)$ is necessary.

Experiment 2: Efficiency across datasets. In this experiment we measure how well our algorithm performs on a variety of different datasets. We use the datasets from Experiment 1, as well as one additional synthetic dataset and 3 datasets with real data. The new synthetic dataset is Sparse random. Each point there is generated by independently sampling each bit to be 1 with probability $1/15$, and to be 0 otherwise. The remaining datasets are: We use embeddings of the data sets *MNIST* [30], *Anonymous Microsoft Web Data* (MSWeb) [12] and *Mushroom* [36, 41] into Hamming space. For MNIST we flatten the images into vectors whose Boolean variables indicate the presence of a pixel. For MSWeb and Mushroom we use one-hot encodings. A more detailed description can be found in the Appendix B.

Due to computational limitations, we restrict the size of each dataset to a maximum of 10000 points. For MNIST and MSWeb, we use only the first 10,000 points, while for the Mushroom dataset, we utilize all 8,124 points. For the synthetic datasets, we generate 10,000 points with a dimension of 300, which is comparable in magnitude to the dimensions of the real datasets. We select parameter

settings where our algorithm has a good chance of success as per the results of Experiment 1. As the number of dimensions varies depending on the dataset, we set $r = 0.15d$. $c = 2$, $\lambda = 4$ and the starting distance is 0 on the second subplot. Otherwise, we use the same setup as Experiment 1, except that for fixed datasets the origin point is the first one in the dataset. On the first subplot the starting distance is measured as a fraction of r. On the second subplot the goal of the algorithm is to find a query to the LSH data structure that returns $\perp$ at a distance from the origin specified by the x axis as a fraction of the cr. The dashed black vertical line represents the radius r and is located at 0.5 since $c = 2$. The results in the first subplot of Figure 2 show that success probability drops monotonically for all datasets as the starting distance grows. This is natural to expect given that similar behaviour was observed in Experiment 1. On the second subplot we can see that for the most datasets we can reliably find a negative query only at distances much larger than r. This, in a way, allows us to compare the "hardness" of datasets, as further away from the origin we move, the less functions collide the origin point z and our current point, the less bits we need to change for the query to become negative. As one can see, all of the real datasets lie between the Random and Sparse random on the second subplot. This can be explained by the fact that the points in real datasets are much sparser than the Random, and the maximum distance from the most isolated point to the nearest point is smaller than the ideal $2cr$.

7 Omitted proofs and statements

This section includes lemmas and proofs omitted in Section 4.

7.1 Proofs from Section 4

Fact 4.6. *For $x \in [0, +\infty)$, $1 - x \le e^{-x}$. For $x \in [0, 1/5]$, $e^{-x-x^2} \le 1 - x$.*

Proof. First we show $e^{-x} - (1-x) \ge 0$. Taking a derivative, we get $-e^{-x}+1$, which is non-negative on $[0, +\infty)$. Therefore, since the inequality holds at 0, it holds on $[0, +\infty)$.

For the second inequality, consider $f(x) := e^{-x-x^2} - (1 - x)$. It is enough to show that it's first derivative $f'(x) = -(2x + 1)e^{-x-x^2} + 1$ is non-positive on $[0, 1/5]$. For this it is sufficient to show that $g(x) := (2x + 1) - e^{x+x^2} \ge 0$ on this interval. $g(x)' = 2 - (2x + 1)e^{x+x^2}$ is a decreasing function. Since $g'(0) > 0$, $g'(1/5) \approx 0.22 > 0$, $g'(x)$ is positive on the interval $[0, 1/5]$. This means that $g(x)$ is non-negative on this interval since $g(0) = 0$. □

Lemma 4.1 (Bounds on k)**.** *If $cr/d \le 1/5$ and $n > e$, one has $2\frac{d}{cr} \ln n \ge k \ge 0.5\frac{d}{cr} \ln n$.*

Proof. Recall that $k = \lceil \frac{\ln n}{-\ln p_2} \rceil = \lceil \ln n \cdot (-\ln(1 - \frac{cr}{d}))^{-1} \rceil$. By taking logarithm of the inequalities of Fact 4.6, we get that for $x \le 1/5$, $x \le -\ln(1 - x) \le x + x^2$. Setting $x = cr/d$, we obtain

$$\ln n \cdot \left(\frac{cr}{d}\right)^{-1} \ge \frac{\ln n}{-\ln(1 - \frac{cr}{d}))} \ge \ln n \cdot \left(\frac{cr}{d} + \left(\frac{cr}{d}\right)^2\right)^{-1}.$$

The required inequalities now follow immediately, since $0 \le cr/d \le 1$ and $\ln n \ge 1$, so $\frac{d}{cr} \ln n \ge 1$ and $\frac{1}{1+cr/d} \le 1/2$. □

7.2 Proofs for Algorithm 3

Here we present Algorithm 3 and its analysis, proving the second part of Theorem 4.9. The key difference compared to Algorithm 2 is the inner loop, which we analyze in a separate lemma.

Lemma 7.1 (Inner loop of the fast adversarial walk)**.** *Let $n > e$, $cr/d \le 1/5$, $c \ge 1 + \frac{80+16\ln(\lambda+1)}{\ln n}$, $8e^2(\lambda + 1) \le n^{1/8}$. Let G be such that no hash functions $g \in G$ has support smaller than $k - 4 \log n$. Suppose that the point z passed to Algorithm 3 is located at distance at least $2cr$ from any other point*

Algorithm 3 Fast adaptive adversarial walk from z.

Input: Origin point z, parameters r, cr, λ, access to an QueryLSH structure

1: $t \leftarrow 2e^2(\lambda+1)$
2: Sample q at distance $r-t$ from z
3: **while** QueryLSH$(q) = z$ **do**
4: **if** $\mathrm{dist}(q,z) \geq r$ **then**
5: **return** $\perp$
6: $q^{left}, q^{right} \leftarrow q$
7: Choose $cr - \mathrm{dist}(q^{right}, z)$ random bits i in q^{right} such that $q_i^{left} \neq z_i$ and flip them
8: **if** QueryLSH$(q^{right}) = z$ **then**
9: **return** $\perp$
10: **while** $\mathrm{dist}(q^{left}, q^{right}) > 1$ **do**
11: $q^{mid} \leftarrow q^{left}$
12: Flip any $\frac{1}{2}\,\mathrm{dist}(q^{left}, q^{right})$ bits i in q^{mid} such that $q_i^{left} \neq q_i^{right}$
13: **if** QueryLSH$(q^{mid}) = z$ **then**
14: $q^{left} \leftarrow q^{mid}$
15: **else**
16: $q^{right} \leftarrow q^{mid}$
17: $j \leftarrow$ the bit where q^{left} differs from q^{right}
18: Flip bit j in q
19: **return** q

in P. Let q sampled in line 2 be such that $|\textsc{Coll}(q,z)| \leq 2e^2(\lambda+1)$. For a fixed iteration of the outer loop in line 3 of Algorithm 3, with probability at least $1 - 1/(32e^2(\lambda+1))$ the inner loop between the lines 6 and 17 finds two points q^{left} and q^{right} such that:

- *QueryLSH$(q^{right}) \neq z$,*
- *QueryLSH$(q^{left}) = z$,*
- *q^{left} and q^{right} differ in one bit.*

The inner loops takes time at most $O(\log(cr - r + t))$.

Proof. Fix an iteration of the outer loop. The idea of the inner loop is to sample two points q^{left} and q^{right} such that there is a path between them that goes strictly away from z and such that QueryLSH$(q^{left}) = z$ and QueryLSH$(q^{right}) \neq z$. Then we move those points towards each other by binary search while preserving those properties.

Initial point q^{right} is sampled through the same process as in Lemma 4.7, so with probability at least $1 - 1/(32e^2(\lambda+1))$ the condition QueryLSH$(q^{right}) \neq z$ holds and the return in line 9 is not reached. The condition QueryLSH$(q^{left}) = z$ holds at the start because QueryLSH$(q) = z$ by the while clause of the outer loop.

During the execution both guarantees continue to hold through the way we reassign variables in the binary search. At the end, the distance between the two points becomes 1, which means that they are differ in exactly one bit. This concludes the correctness proof.

The running time is $O(\log(cr - (r-t))$ because the distance between q^{left} and q^{right} is halved after each iteration. □

The proof of Algorithm 3 is almost the same as of Algorithm 2, so we only outline the differences.

Theorem 7.2 (Fast algorithm). *Let $n > e$, $28\ln^3 n \le r \le d/(28\ln n)$, $\lambda \le \frac{1}{8e^2}\min\{\frac{r}{\ln n}, n^{1/8}\} - 1$, and $1 + \frac{80+16\ln(\lambda+1)}{\ln n} \le c \le \ln n$. Let $t = 2e^2(\lambda+1)$ be the parameter from Algorithm 2 (Algorithm 3). Suppose that the point z passed to Algorithm 2 (Algorithm 3) is located at a distance of at least $2cr$ from any other point in P. With probability at least $1/4 - 1/n$ the algorithm finds a point q at a distance of at most r from z such that querying LSH with q returns no point. It uses at most $O(cr \cdot \lambda)$ ($O(\log(cr) \cdot \lambda)$ for Algorithm 3) queries to the LSH data structure.*

Proof. Algorithms 2 and 3 only differ in the implementation of the inner loop, between the lines 6 and 12 and the lines 6 and 17 respectively. Because of this, the proof is exactly the same as in Algorithm 2, however Lemma 7.1 is used instead of Lemma 4.8.

The final runtime is $O(\log(cr - r + t) \cdot \lambda)$ since the inner loop takes time $O(\log(cr - r + t))$. □

7.3 Existence of an isolated point

Not all data sets contain a sufficiently isolated point as required by Theorem 4.9, i.e. a point that is at a distance $2cr$ from any other point in the set. However, when $2cr < d/4$ and the point set is sampled randomly, each point in the data set will be isolated with high probability.

Lemma 7.3. *Let P be a set of n random points sampled uniformly independently at random from $\{0,1\}^d$. If $d \ge 48\ln n$, with probability at least $1 - 1/n$ the distance between any two of them is at least $d/4$.*

Proof. Fix an arbitrary point z. We will first show that the probability that $\mathrm{dist}(z,x) < d/4$ is at most $1/n^3$ for a uniformly random point x.

Notice that $\mathrm{dist}(z,x) = \sum_{i=1}^d y_i$, where $y_i = |z_i - x_i|$, and all y_i are independent with respect to each other and are equal to 1 with probability $1/2$ and 0 otherwise. Hence $\mathbb{E}[\sum_{i=1}^d y_i] = d/2$. Therefore, by the Chernoff bound,

$$\Pr[\mathrm{dist}(z,x) \le 0.5 \cdot 0.5d] \le e^{-\frac{0.25\cdot 0.5d}{2}} \le e^{-3\ln n} = 1/n^3.$$

Now, by a union bound across all pairs of points, the probability that the minimum of pairwise distances between any two of them is at most $d/4$ is at most $1/n$. □

References

[1] Thomas Dybdahl Ahle. 2017. Optimal Las Vegas Locality Sensitive Data Structures. In *58th IEEE Annual Symposium on Foundations of Computer Science, FOCS 2017, Berkeley, CA, USA, October 15-17, 2017*, Chris Umans (Ed.). IEEE Computer Society, 938–949. https://doi.org/10.1109/FOCS.2017.91

[2] Alexandr Andoni and Piotr Indyk. 2006. Near-Optimal Hashing Algorithms for Approximate Nearest Neighbor in High Dimensions. In *47th Annual IEEE Symposium on Foundations of Computer Science (FOCS 2006), 21-24 October 2006, Berkeley, California, USA, Proceedings*. IEEE Computer Society, 459–468. https://doi.org/10.1109/FOCS.2006.49

[3] Alexandr Andoni, Piotr Indyk, Thijs Laarhoven, Ilya P. Razenshteyn, and Ludwig Schmidt. 2015. Practical and Optimal LSH for Angular Distance. In *Advances in Neural Information Processing Systems 28: Annual Conference on Neural Information Processing Systems 2015, December 7-12, 2015, Montreal, Quebec, Canada*, Corinna Cortes, Neil D. Lawrence, Daniel D. Lee, Masashi Sugiyama, and Roman Garnett (Eds.). 1225–1233. https://proceedings.neurips.cc/paper/2015/hash/2823f4797102ce1a1aec05359cc16dd9-Abstract.html

[4] Alexandr Andoni, Piotr Indyk, Huy L. Nguyen, and Ilya P. Razenshteyn. 2014. Beyond Locality-Sensitive Hashing. In *Proceedings of the Twenty-Fifth Annual ACM-SIAM Symposium on Discrete Algorithms, SODA 2014, Portland, Oregon, USA, January 5-7, 2014*, Chandra Chekuri (Ed.). SIAM, 1018–1028. https://doi.org/10.1137/1.9781611973402.76

[5] Alexandr Andoni, Thijs Laarhoven, Ilya P. Razenshteyn, and Erik Waingarten. 2017. Optimal Hashing-based Time-Space Trade-offs for Approximate Near Neighbors. In *Proceedings of the Twenty-Eighth Annual ACM-SIAM Symposium on Discrete Algorithms, SODA 2017, Barcelona, Spain, Hotel Porta Fira, January 16-19*, Philip N. Klein (Ed.). SIAM, 47–66. https://doi.org/10.1137/1.9781611974782.4

[6] Alexandr Andoni and Ilya P. Razenshteyn. 2015. Optimal Data-Dependent Hashing for Approximate Near Neighbors. In *Proceedings of the Forty-Seventh Annual ACM on Symposium on Theory of Computing, STOC 2015, Portland, OR, USA, June 14-17, 2015*, Rocco A. Servedio and Ronitt Rubinfeld (Eds.). ACM, 793–801. https://doi.org/10.1145/2746539.2746553

[7] Idan Attias, Edith Cohen, Moshe Shechner, and Uri Stemmer. 2023. A Framework for Adversarial Streaming via Differential Privacy and Difference Estimators. In *14th Innovations in Theoretical Computer Science Conference, ITCS 2023, January 10-13, 2023, MIT, Cambridge, Massachusetts, USA (LIPIcs, Vol. 251)*, Yael Tauman Kalai (Ed.). Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 8:1–8:19. https://doi.org/10.4230/LIPICS.ITCS.2023.8

[8] Martin Aumüller, Tobias Christiani, Rasmus Pagh, and Francesco Silvestri. 2018. Distance-Sensitive Hashing. In *Proceedings of the 37th ACM SIGMOD-SIGACT-SIGAI Symposium on Principles of Database Systems, Houston, TX, USA, June 10-15, 2018*, Jan Van den Bussche and Marcelo Arenas (Eds.). ACM, 89–104. https://doi.org/10.1145/3196959.3196976

[9] Amos Beimel, Haim Kaplan, Yishay Mansour, Kobbi Nissim, Thatchaphol Saranurak, and Uri Stemmer. 2022. Dynamic algorithms against an adaptive adversary: generic constructions and lower bounds. In *STOC '22: 54th Annual ACM SIGACT Symposium on Theory of Computing, Rome, Italy, June 20 - 24, 2022*, Stefano Leonardi and Anupam Gupta (Eds.). ACM, 1671–1684. https://doi.org/10.1145/3519935.3520064

[10] Omri Ben-Eliezer, Rajesh Jayaram, David P. Woodruff, and Eylon Yogev. 2022. A Framework for Adversarially Robust Streaming Algorithms. *J. ACM* 69, 2 (2022), 17:1–17:33. https://doi.org/10.1145/3498334

[11] Omri Ben-Eliezer and Eylon Yogev. 2020. The Adversarial Robustness of Sampling. In *Proceedings of the 39th ACM SIGMOD-SIGACT-SIGAI Symposium on Principles of Database Systems, PODS 2020, Portland, OR, USA, June 14-19, 2020*, Dan Suciu, Yufei Tao, and Zhewei Wei (Eds.). ACM, 49–62. https://doi.org/10.1145/3375395.3387643

[12] Jack Breese, David Heckerman, and Carl Kadie. 1998. Anonymous Microsoft Web Data. UCI Machine Learning Repository. DOI: https://doi.org/10.24432/C5VS3Q.

[13] Moses Charikar. 2002. Similarity estimation techniques from rounding algorithms. In *Proceedings on 34th Annual ACM Symposium on Theory of Computing, May 19-21, 2002, Montréal, Québec, Canada*, John H. Reif (Ed.). ACM, 380–388. https://doi.org/10.1145/509907.509965

[14] Yeshwanth Cherapanamjeri and Jelani Nelson. 2020. On Adaptive Distance Estimation. In *Advances in Neural Information Processing Systems 33: Annual Conference on Neural Information Processing Systems 2020, NeurIPS 2020, December 6-12, 2020, virtual*, Hugo Larochelle, Marc'Aurelio Ranzato, Raia Hadsell, Maria-Florina Balcan, and Hsuan-Tien Lin (Eds.). https://proceedings.neurips.cc/paper/2020/hash/803ef56843860e4a48fc4cdb3065e8ce-Abstract.html

[15] Yeshwanth Cherapanamjeri, Sandeep Silwal, David P. Woodruff, Fred Zhang, Qiuyi Zhang, and Samson Zhou. 2023. Robust Algorithms on Adaptive Inputs from Bounded Adversaries. In *The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1-5, 2023*. OpenReview.net. https://openreview.net/forum?id=I29Kt0RwChs

[16] Tobias Christiani. 2017. A Framework for Similarity Search with Space-Time Tradeoffs using Locality-Sensitive Filtering. In *Proceedings of the Twenty-Eighth Annual ACM-SIAM Symposium on Discrete Algorithms, SODA 2017, Barcelona, Spain, Hotel Porta Fira, January 16-19*, Philip N. Klein (Ed.). SIAM, 31–46. https://doi.org/10.1137/1.9781611974782.3

[17] Edith Cohen, Xin Lyu, Jelani Nelson, Tamás Sarlós, Moshe Shechner, and Uri Stemmer. 2022. On the Robustness of CountSketch to Adaptive Inputs. In *International Conference on Machine Learning, ICML 2022, 17-23 July 2022, Baltimore, Maryland, USA (Proceedings of Machine Learning Research, Vol. 162)*, Kamalika Chaudhuri, Stefanie Jegelka, Le Song, Csaba Szepesvári, Gang Niu, and Sivan Sabato (Eds.). PMLR, 4112–4140. https://proceedings.mlr.press/v162/cohen22a.html

[18] Edith Cohen, Jelani Nelson, Tamás Sarlós, and Uri Stemmer. 2023. Tricking the Hashing Trick: A Tight Lower Bound on the Robustness of CountSketch to Adaptive Inputs. In *Thirty-Seventh AAAI Conference on Artificial Intelligence, AAAI 2023, Thirty-Fifth Conference on Innovative Applications of Artificial Intelligence, IAAI 2023, Thirteenth Symposium on Educational Advances in Artificial Intelligence, EAAI 2023, Washington, DC, USA, February 7-14, 2023*, Brian Williams, Yiling Chen, and Jennifer Neville (Eds.). AAAI Press, 7235–7243. https://doi.org/10.1609/AAAI.V37I6.25882

[19] Søren Dahlgaard, Mathias Bæk Tejs Knudsen, and Mikkel Thorup. 2017. Practical Hash Functions for Similarity Estimation and Dimensionality Reduction. In *Advances in Neural Information Processing Systems 30: Annual Conference on Neural Information Processing Systems 2017, December 4-9, 2017, Long Beach, CA, USA*, Isabelle Guyon, Ulrike von Luxburg, Samy Bengio, Hanna M. Wallach, Rob Fergus, S. V. N. Vishwanathan, and Roman Garnett (Eds.). 6615–6625. https://proceedings.neurips.cc/paper/2017/hash/62dad6e273d32235ae02b7d321578ee8-Abstract.html

[20] Mayur Datar, Nicole Immorlica, Piotr Indyk, and Vahab S Mirrokni. 2004. Locality-sensitive hashing scheme based on p-stable distributions. In *Proceedings of the twentieth annual symposium on Computational geometry*. 253–262.

[21] Elena Gribelyuk, Honghao Lin, David P. Woodruff, Huacheng Yu, and Samson Zhou. 2024. A Strong Separation for Adversarially Robust ℓ_0 Estimation for Linear Sketches. In *65th IEEE Annual Symposium on Foundations of Computer Science, FOCS 2024, Chicago, IL, USA, October 27-30, 2024*. IEEE, 2318–2343. https://doi.org/10.1109/FOCS61266.2024.00136

[22] Moritz Hardt and David P. Woodruff. 2013. How robust are linear sketches to adaptive inputs?. In *Symposium on Theory of Computing Conference, STOC'13, Palo Alto, CA, USA, June 1-4, 2013*, Dan Boneh, Tim Roughgarden, and Joan Feigenbaum (Eds.). ACM, 121–130. https://doi.org/10.1145/2488608.2488624

[23] Avinatan Hassidim, Haim Kaplan, Yishay Mansour, Yossi Matias, and Uri Stemmer. 2022. Adversarially Robust Streaming Algorithms via Differential Privacy. *J. ACM* 69, 6 (2022), 42:1–42:14. https://doi.org/10.1145/3556972

[24] Piotr Indyk and Rajeev Motwani. 1998. Approximate Nearest Neighbors: Towards Removing the Curse of Dimensionality. In *Proceedings of the Thirtieth Annual ACM Symposium on the Theory of Computing, Dallas, Texas, USA, May 23-26, 1998*, Jeffrey Scott Vitter (Ed.). ACM, 604–613. https://doi.org/10.1145/276698.276876

[25] Omid Jafari, Preeti Maurya, Parth Nagarkar, Khandker Mushfiqul Islam, and Chidambaram Crushev. 2021. A survey on locality sensitive hashing algorithms and their applications. *arXiv preprint arXiv:2102.08942* (2021).

[26] Michael Kapralov. 2015. Smooth Tradeoffs between Insert and Query Complexity in Nearest Neighbor Search. In *Proceedings of the 34th ACM Symposium on Principles of Database Systems, PODS 2015, Melbourne, Victoria, Australia, May 31 - June 4, 2015*, Tova Milo and Diego Calvanese (Eds.). ACM, 329–342. https://doi.org/10.1145/2745754.2745761

[27] Christopher Kennedy and Rachel A. Ward. 2017. Fast Cross-Polytope Locality-Sensitive Hashing. In *8th Innovations in Theoretical Computer Science Conference, ITCS 2017, January 9-11, 2017, Berkeley, CA, USA (LIPIcs, Vol. 67)*, Christos H. Papadimitriou (Ed.). Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 53:1–53:16. https://doi.org/10.4230/LIPICS.ITCS.2017.53

[28] Hisashi Koga, Tetsuo Ishibashi, and Toshinori Watanabe. 2007. Fast agglomerative hierarchical clustering algorithm using Locality-Sensitive Hashing. *Knowledge and Information Systems* 12 (2007), 25–53.

[29] Eyal Kushilevitz, Rafail Ostrovsky, and Yuval Rabani. 2000. Efficient Search for Approximate Nearest Neighbor in High Dimensional Spaces. *SIAM J. Comput.* 30, 2 (2000), 457–474. https://doi.org/10.1137/S0097539798347177

[30] Yann LeCun. 1998. The MNIST database of handwritten digits. *http://yann. lecun. com/exdb/mnist/* (1998).

[31] Ping Li, Art B. Owen, and Cun-Hui Zhang. 2012. One Permutation Hashing. In *Advances in Neural Information Processing Systems 25: 26th Annual Conference on Neural Information Processing Systems 2012. Proceedings of a meeting held December 3-6, 2012, Lake Tahoe, Nevada, United States*, Peter L. Bartlett, Fernando C. N. Pereira, Christopher J. C. Burges, Léon Bottou, and Kilian Q. Weinberger (Eds.). 3122–3130. https://proceedings.neurips.cc/paper/2012/hash/eaa32c96f620053cf442ad32258076b9-Abstract.html

[32] Jelani Nelson, Huy L. Nguyên, and David P. Woodruff. 2012. On Deterministic Sketching and Streaming for Sparse Recovery and Norm Estimation. In *Approximation, Randomization, and Combinatorial Optimization. Algorithms and Techniques - 15th International Workshop, APPROX 2012, and 16th International Workshop, RANDOM 2012, Cambridge, MA, USA, August 15-17, 2012. Proceedings (Lecture Notes in Computer Science, Vol. 7408)*, Anupam Gupta, Klaus Jansen, José D. P. Rolim, and Rocco A. Servedio (Eds.). Springer, 627–638. https://doi.org/10.1007/978-3-642-32512-0_53

[33] Rasmus Pagh. 2016. Locality-sensitive Hashing without False Negatives. In *Proceedings of the Twenty-Seventh Annual ACM-SIAM Symposium on Discrete Algorithms, SODA 2016, Arlington, VA, USA, January 10-12, 2016*, Robert Krauthgamer (Ed.). SIAM, 1–9. https://doi.org/10.1137/1.9781611974331.ch1

[34] Rina Panigrahy. 2006. Entropy based nearest neighbor search in high dimensions. In *Proceedings of the Seventeenth Annual ACM-SIAM Symposium on Discrete Algorithms, SODA 2006, Miami, Florida, USA, January 22-26, 2006*. ACM Press, 1186–1195. http://dl.acm.org/citation.cfm?id=1109557.1109688

[35] Matti Ryynanen and Anssi Klapuri. 2008. Query by humming of midi and audio using locality sensitive hashing. In *2008 IEEE International Conference on Acoustics, Speech and Signal Processing*. 2249–2252. https://doi.org/10.1109/ICASSP.2008.4518093

[36] UC Irvine Machine Learning Repository 1987. Mushroom. UCI Machine Learning Repository. DOI: https://doi.org/10.24432/C5959T.

[37] Roger Weber, Hans-Jörg Schek, and Stephen Blott. 1998. A Quantitative Analysis and Performance Study for Similarity-Search Methods in High-Dimensional Spaces. In *VLDB'98, Proceedings of 24rd International Conference on Very Large Data Bases, August 24-27, 1998, New York City, New York, USA*, Ashish Gupta, Oded Shmueli, and Jennifer Widom (Eds.). Morgan Kaufmann, 194–205. http://www.vldb.org/conf/1998/p194.pdf

[38] Alexander Wei. 2022. Optimal Las Vegas Approximate Near Neighbors in ℓ_p. *ACM Trans. Algorithms* 18, 1 (2022), 7:1–7:27. https://doi.org/10.1145/3461777

[39] David Woodruff and Samson Zhou. 2024. Adversarially Robust Dense-Sparse Tradeoffs via Heavy-Hitters. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*.

[40] David P. Woodruff and Samson Zhou. 2021. Tight Bounds for Adversarially Robust Streams and Sliding Windows via Difference Estimators. In *62nd IEEE Annual Symposium on Foundations of Computer Science, FOCS 2021, Denver, CO, USA, February 7-10, 2022*. IEEE, 1183–1196. https://doi.org/10.1109/FOCS52979.2021.00116

[41] Barry L Wulff. 1982. The Audubon Society Field Guide to North American Mushrooms, by Gary Lincoff.

[42] Ying Zhang, Huchuan Lu, Lihe Zhang, Xiang Ruan, and Shun Sakai. 2016. Video anomaly detection based on locality sensitive hashing filters. *Pattern Recognition* 59 (2016), 302–311.

A Dense datasets are robust

Our main result requires that the starting point is isolated, i.e. the other closest point in the dataset is located at a distance at least $2cr$. This is a rather restrictive condition. However, in this section we provide some evidence on why some degree of sparsity in the dataset is required for the false negative queries to exist at all.

Namely, we will show that LSH perfectly hashes any two points at a distance slightly smaller than αr, for a constant $\alpha < 1$, in some parameter settings. This is achieved through a more careful version of the analysis from Indyk and Motwani [24]. This implies that if the dataset is very dense, i.e. for any possible query there exists a point very close to it, the LSH scheme as described in Section 3 cannot return a false negative point. However, this variation might have the same weakness against an adaptive adversary as other false-negative-free LSH constructions, which is that the running time to answer a given query is bounded only by the size of the dataset. It is an intriguing open problem to investigate whether an attack can be constructed that can find such a query quickly and reliably.

The exact claim that we make is as follows:

Lemma A.1. *Suppose that c is a constant, $s < \min\{\alpha r, \frac{1}{2}\lambda \cdot n^{\frac{2(1-\alpha)}{5c}} (\ln d)^{-1}\}$ for a constant $0 < \alpha < 1$ and $r < d/5$. Then for any point p in the dataset, under the randomness of G, any query within distance s to p will not be a false-negative with high probability for an LSH data structure constructed according to Section 3.*

Proof. Following the same logic as in Lemma 4.5, for a fixed p, for any given q the probability that q and p collide under a randomly chosen hash function g is

$$\Pr_g[g(q) = g(p)] = \left(1 - \frac{\operatorname{dist}(q,p)}{d}\right)^k.$$

Hence, by union bound, the probability that none of the functions in G hash p and q together is

$$\Pr_G[\forall g \in G : g(q) \neq g(p)] \leq \left(1 - \left(1 - \frac{\operatorname{dist}(q,p)}{d}\right)^k\right)^L.$$

By Fact 4.6, and since $L = \lceil \lambda \ell \rceil$, we can upper bound this value by

$$\left(1 - \left(1 - \frac{\operatorname{dist}(q,p)}{d}\right)^k\right)^L \leq \exp\left(-\left(1 - \frac{\operatorname{dist}(q,p)}{d}\right)^k\right)^L = \exp\left(-\left(1 - \frac{\operatorname{dist}(q,p)}{d}\right)^k \cdot L\right) \leq$$
$$\exp\left(-\lambda\left(1 - \frac{\operatorname{dist}(q,p)}{d}\right)^k \left(1 - \frac{r}{d}\right)^{-k}\right) = \exp\left(-\lambda\left(\frac{d - \operatorname{dist}(q,p)}{d - r}\right)^k\right) =$$
$$\exp\left(-\lambda\left(\frac{d - \operatorname{dist}(q,p)}{d - r}\right)^k\right) = \exp\left(-\lambda\left(1 + \frac{r - \operatorname{dist}(q,p)}{d - r}\right)^k\right).$$

We will now bound the argument of the exponent, again using Fact 4.6 and Lemma 4.1.

$$\left(1+\frac{r-\mathrm{dist}(q,p)}{d-r}\right)^k \geq \left(1+\frac{r-\mathrm{dist}(q,p)}{d}\right)^k \geq \exp\left[k\cdot\left(\frac{r-\mathrm{dist}(q,p)}{d}-\left(\frac{r-\mathrm{dist}(q,p)}{d}\right)^2\right)\right] \geq$$
$$\exp\left[\left(0.5\frac{d}{cr}\ln n\right)\cdot\frac{r-\mathrm{dist}(q,p)}{d}\left(1-\frac{r-\mathrm{dist}(q,p)}{d}\right)\right] =$$
$$\exp\left[0.5\frac{\ln n}{c}\cdot\frac{r-\mathrm{dist}(q,p)}{r}\left(1-\frac{r-\mathrm{dist}(q,p)}{d}\right)\right] \geq n^{\frac{2}{5}\cdot\frac{1}{c}\cdot\left(1-\frac{\mathrm{dist}(q,p)}{r}\right)}.$$

Hence, we get the bound of

$$\Pr_G[\forall g\in G: g(q)\neq g(p)] \leq \exp\left(-\lambda\cdot n^{\frac{2}{5}\cdot\frac{1}{c}\cdot\left(1-\frac{\mathrm{dist}(q,p)}{r}\right)}\right).$$

Denote by Q the set of all points lying at distance at most s from p. Then $|Q|\leq d^s$. By union bound, and bounds on s we get:

$$\Pr_G[\exists q\in Q:\forall g\in G: g(q)\neq g(p)] \leq d^s\cdot\exp\left(-\lambda\cdot n^{\frac{2}{5}\cdot\frac{1}{c}\cdot\left(1-\frac{s}{r}\right)}\right) \leq \exp\left(-0.5\lambda\cdot n^{\frac{2}{5}\cdot\frac{1}{c}\cdot(1-\alpha)}\right).$$

Since this equation states that the probability that any point in Q is a false-negative is exponentially small in n, we can conlude the claim of the lemma.

□

Note that by choosing any α, we can then calculate s and choose r to be equal to s/α. In this way, we can easily construct an instance where points closer than αr to a point in the dataset are not false negatives.

In this setting, if the dataset is dense enough such that any possible query is within αr distance to the nearest point in the dataset, then no false negatives exist.

B Additional experiments and details

Resources used. The experiments were run on a server in an internal cluster, where a typical worker has two AMD EPYC 7302 16-Core processors, with up to 500 GiB of RAM. Experiments where run in parallel on multiple cores. In total, running all of the experiments, including those in the appendix, takes at most 100000 CPU hours, and uses 21 GiB of disk memory.

Data sets used in experiment 2. MNIST [30]. This dataset consists of grayscale pictures of handwritten digits of size 28×28. We first flatten the images into vectors of size 784, and then we project those vectors into Hamming space by setting each entry to 0 if it corresponded to a white pixel, i.e. if the grayscale color was 0, and to 1 otherwise. Note that this transformation preserves the dataset fairly well, as the background pixels in each image are perfectly white.

Anonymous Microsoft Web Data (MSWeb) [12]. Entries in this dataset are anonymous web user IDs and paths on the Microsoft website that they have visited. There are in total 294 different paths. We represent each user by a point in 294 dimensional Hamming space, where a point entry is equal to 1 if they have visited the corresponding path.

Mushroom [36, 41]. This dataset consists of descriptions of samples of mushrooms of 23 different species. Each mushroom has 22 categorical features describing it. We one-hot encode each feature to map the descriptions into Hamming space, with the final number of dimensions being 116.

Data sets distances. When we set $r=0.15d$ and $cr=0.3d$, their values depend on the dimension of the points of the dataset. For reference, in Table 1 we present their values, compared with mean distance to the nearest neighbor.

Table 1. Values of r, cr and mean nearest neighbor distance for each dataset.

Dataset	r	cr	mean 1NN
MSWeb	44	88	0.75
MNIST	117	234	41.06
Mushroom	17	34	2.0
Random	45	90	115.76
Sparse	45	90	53.91

Experiment 3: Influence of λ on the algorithm efficiency. In this experiment the goal was to measure the impact of λ on the probability of success of the algorithm. The setup is the same as in Experiment 2. All runs are done on the Random dataset. The results are on Figure 3.

Similar to Experiment 2, we compare the different values of λ by varying the distance at which we want to find the negative query. As is evident from the plots, the bigger the λ, the harder it is for the algorithm to succeed.

Note that the value of λ where algorithm no longer succeeds for distance r are rather modest. It may be that it is enough to just increase the value of λ to defend against the type of attack that is described in this paper.

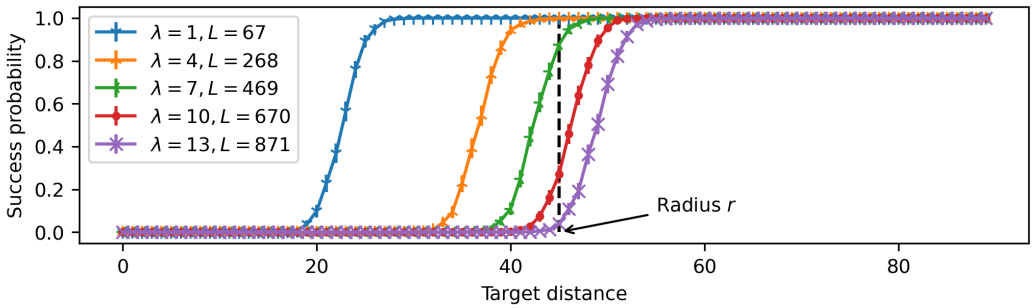

Fig. 3. Desired distance of the query not hashing with the origin point from the origin point z for different values of λ. The experiments are conducted on the Zero dataset for the default parameter setting. The distance equal to r is denoted by a dashed vertical line.

Experiment 4: Number of queries as a function of starting distance. In this experiment we experimentally confirm that the number of queries that an algorithm needs is linearly dependent on t. We use the same datasets and experimental setting as in Experiment 2. The results are on Figure 5.

Combined with results of Experiment 2, we can see that lowering the starting distance provides a trade-off between the running time and the probability of success.

Experiment 5: Efficiency gain over random sampling. Finally, we investigated the efficiency gain that our adaptive algorithm achieved over random point sampling. The experimental setup is the same as in Experiment 2. Here we use the Random dataset. For both approaches, we continued rerunning them until a false negative query was found. The results are presented in Figure 4.

As you can see, when the λ and, accordingly, the number of hash functions L are very small, random sampling is more efficient. However, for even modest values of λ, we already get a significant exponential speed-up.

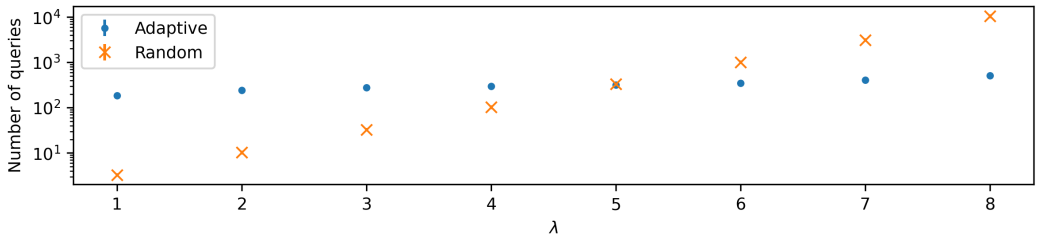

Fig. 4. Mean number of queries necessary to find a false negative depending on parameter λ required by our adaptive adversary and random sampling of query points.

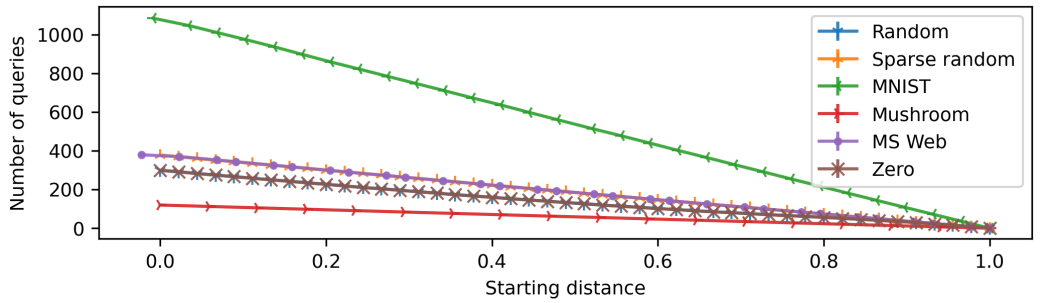

Fig. 5. The number of queries as a function of the distance of q from origin point z on all tested datasets.

Experiment 6: Comparison of defense mechanisms. In this experiment we compare two different defence mechanisms based on ideas from differential privacy [9] and distance estimation [14].

Theorem 1.3. of [14] presents a general robustification mechanism which, when applied to then LSH of [24], says that it can be made robust with blow-ups of $O(d \log 1/\delta)$ for memory and $O(\log 1/\delta)$ for query time, where δ is the failure probability per adaptive query. This is achieved by creating $O(d \log 1/\delta)$ copies of plain LSH. Querying is done by selecting $O(\log 1/\delta)$ of those LSH data structures, relaying the query to them and then choosing any of the returned points as the answer, or $\perp$ if all of the data structure failed to find a point.

As a side note, the original ADE construction of [14] would also allow to solve the ANN problem. We don't study it in this experiment for two reasons: it would require time per query $\Omega(n)$, which is too slow in our parameter setting, and it is based upon linear sketches, against which our adaptive adversary is not effective.

The generic differentially private conversion of [9] is similar. For a generic estimation data structure, to be able to process T queries against an adaptive adversary, they create $\widetilde{O}(\sqrt{T})$ copies of the data structure. For each query, they select $\widetilde{O}(1)$ copies of the data structure and aggregate their output using a differentially private median. Although their construction is not generally applicable to ANN, as it is a search problem, it would nevertheless still be effective against our adversary, as for the latter it is not important which exact point was returned, but only whether anything was returned at all.

As both of the data structures can provably defend against an adaptive adversary and because both of them require a lot of additional memory, in this experiment we consider their performance against our adversary when the available memory is limited. Similarly, because rigorous implementation of

a differentially private median would be computationally expensive, we instead opt to use a slighlty simplified version adapted to our setting. Namely, after sampling and querying the data structures, we count the number of them that returned a point and those that did not. We add to both values two-sided geometric noise, whose probability mass function is$\Pr[z] = \frac{1-\alpha}{1+\alpha}\alpha^{|z|}$, with parameter $\alpha = e^{-1/4}$. If after this perturbation the number of data structures that returned $\perp$ is bigger than the ones that returned a point, we return $\perp$. Otherwise, one of the outputs of the successful data structures is returned.

The results are depicted on Figures 6 and 7. On both figures, plots annotated with both λ and "q.s." correspond to one of the two robustified variation accordingly. λ represents the number of copies used, and "q.s." stands for "query samples", and represents the number of data structures randomly sampled to answer each query. Each used copy is build with the same parameters as in experiment 2, and $\lambda = 1$. The plots annotated with only λ are the plain LSH constructions with different values of λ. The idea is that the plots with the same value of λ have the same number of initialized hash functions. The used dataset is Random.

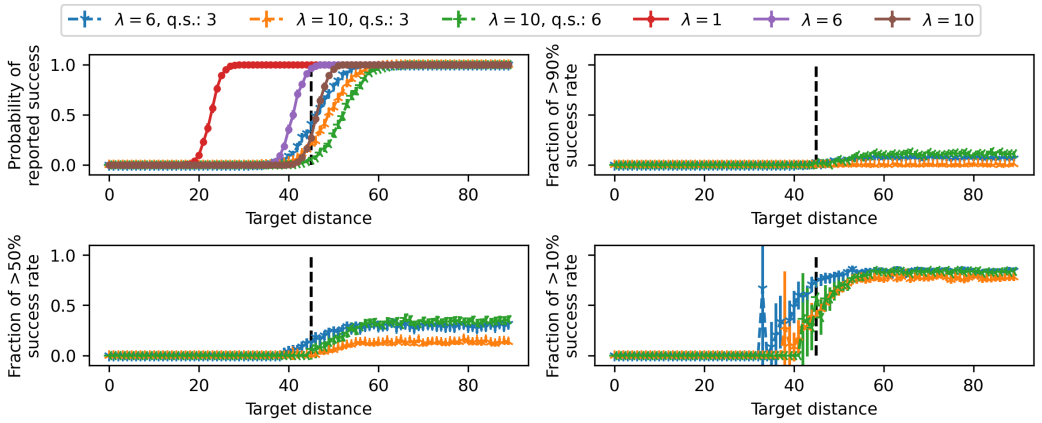

Fig. 6. Performance of robustification based on [14]. First subplot depicts probability that our adaptive adversary finds a false negative query versus the desired distance to that query from origin. The rest of subplots depict probabilities that, if such a query was found, if it was queried again it would still be a false negative query with probability at least 90%, 50% or 10%. In the legend, plots annotated with both λ and "q.s" correspond to the robust algorithm, with "q.s." standing for "query samples", the number of data structures sampled to answer each query, and λ representing the total number of used data structures. The rest are the plain LSH construction with different values of λ.

Both figures contain 4 subplots. The first subplot represents the probability that our adaptive adversary reports that it has found a false negative query at a desired target distance. Note that here, unlike in our previous experiments, the output of each query is non-deterministic even after the randomness of the hash functions is fixed. Hence we also test how "good" of a false negative the reported point is. For each found point, we repeatedly query it to the data structure 100 times, and depict on the other three subplots the probability of the points discovered by the adversary to still be a negative query for at least 90%, 50% and 10% of the repeated queries.

As you can see from the Figure 6, the defense based on [14] is performing better than querying the plain LSH with the same and even often greater amount of hash functions. Similarly, the probability that the discovered false negative remain false negatives in at least 50% or 90% of cases is fairly low.

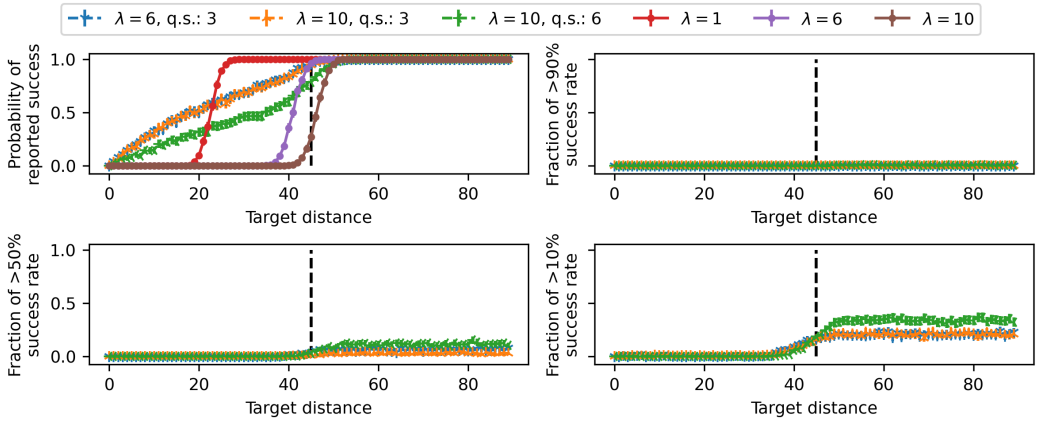

Fig. 7. Performance of robustification based on differential privacy [9]. First subplot depicts probability that our adaptive adversary finds a false negative query versus the desired distance to that query from origin. The rest of subplots depict probabilities that, if such a query was found, if it was queried again it would still be a false negative query with probability at least 90%, 50% or 10%. In the legend, plots annotated with both λ and "q.s" correspond to the robust algorithm, with "q.s." standing for "query samples", the number of data structures sampled to answer each query, and λ representing the total number of used data structures. The rest are the plain LSH construction with different values of λ.

On the other hand, the results for the algorithm based on differential privacy on Figure 7 show that the probability of reporting a false negative is considerably higher than for a plain LSH even for low values of target distance. This is due to the aggregation mechanism that we use, as it has a chance of reporting nothing even if all sampled data structures returned a positive answer. However, this also means that the adversary has less chances to learn the underlying data structure, which we can see in the fact that the probability of that a repeated query would be a negative one in at least 10% of cases being below 0.5.

Based on this, it appears that unless in your application you are prepared to tolerate a significantly higher chance of failure for each query, it is better to use the construction based on [14]. Otherwise, a differentially private construction would leak less information to the adversary. Note that both constructions are still computationally faster than directly querying each data structure.

Received December 2024; revised February 2025; accepted March 2025